Estructuras de Datos

SC-304

Integrantes:

Edwin Pardo García

Ignacio Vargas Mendoza

Dereck Andrés Villalobos Vargas

Sergio Hernández Chavarría

I Avance de proyecto

Profesor:

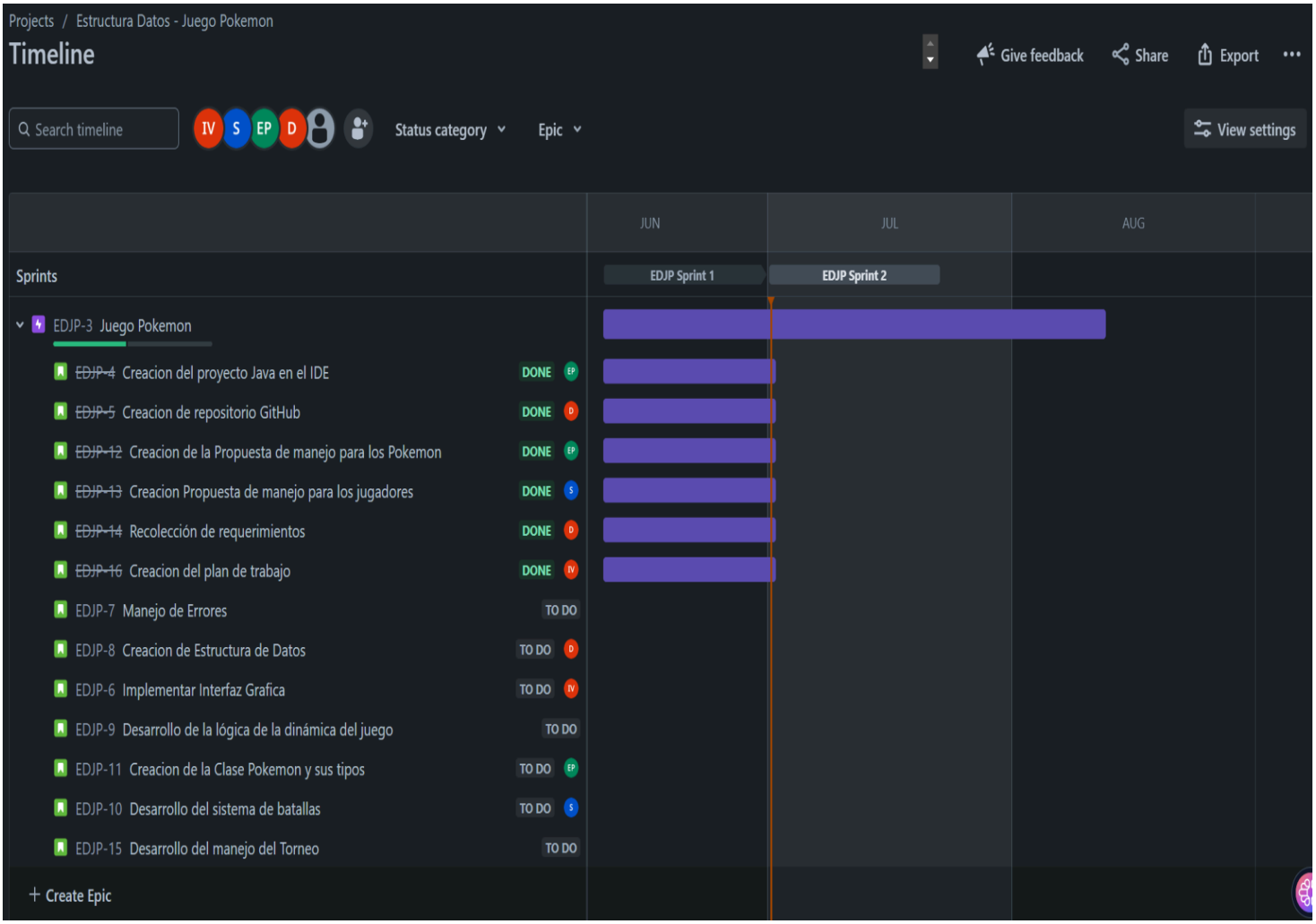
Pablo Gómez Ramírez

Cuatrimestre:

(II Cuatrimestre)

Plan de trabajo de proyecto

https://ignaciov.atlassian.net/jira/software/projects/EDJP/boards/3/timeline?shared=&atlOrigin=eyJpIjoiYzUyNWEyNTM3NzQ4NDkzZjgxODNhYWU5OGUxMTZlNzAiLCJwIjoiaiJ9



Recolección de requerimientos

Requerimientos generales

- **Lenguaje de programación:** Java.
- **Control de versiones:** GitHub con commits frecuentes.
- **Interfaz gráfica:** No utilizar JOptionPane ni consola.
- **Ejecución sin errores:** El proyecto debe funcionar sin errores.
- **Estructuras de datos:** Se prohíbe el uso de ArrayLists, arreglos de Java y estructuras de datos de la biblioteca java.utils. Se deben crear todas las estructuras en el proyecto.
- **Cantidad de estructuras de datos:** El proyecto debe contener al menos 3 estructuras de datos vistas en clase.

Requerimientos específicos

Objetivo del juego: Convertirse en el Maestro Pokémon Absoluto.

Dinámica:

- El jugador debe ganar tres batallas contra maestros Pokémon controlados por el programa.
- El jugador gana la batalla si todos los Pokémon del oponente quedan fuera de combate (knocked-out).
- El jugador pierde la batalla si todos sus Pokémon quedan fuera de combate (knocked-out).
- Existen 3 Pokémon de cada tipo (Normal, Fuego y Agua) para un total de 9 Pokémon.

Torneo:

- El torneo consta de 3 peleas para el jugador.
- El progreso en el torneo debe mostrarse antes de cada batalla.
- Las llaves con solo oponentes de CPU no contienen batallas, el ganador se puede elegir aleatoriamente.

Batalla:

- Participantes: Dos jugadores (jugador y CPU).
- Objetivo: Ganar la batalla haciendo que el oponente se quede sin Pokémon.

Selección de Pokémon:

- El jugador comienza con 4 Pokémon y debe elegirlos de su Pokedex. No puede haber Pokémon repetidos.
- El oponente (CPU) se le asigna 4 Pokémon aleatorios al inicio de la batalla. Cada Pokémon puede repetirse solo una vez en el Pokedex del oponente.
- Desarrollo de la batalla:
- Los Pokémon no pueden utilizarse en la batalla si se acaba su vida (HP).

- Los ataques pueden ser bloqueados con defensa o defensa especial.
- Los ataques especiales tardan 2 turnos en recargarse.
- Las defensas especiales tardan 2 turnos en recargarse.
- Los ataques tienen la posibilidad de fallar.
- El jugador puede cambiar su Pokémon por otro durante la batalla. El Pokémon reemplazado mantiene los HP que tenía hasta el momento de su cambio.
- El oponente (CPU) no puede reemplazar Pokémon.

Propuesta de manejo para los Pokemon

Para manejar los Pokémon se utilizará la estructura de pilas, esto principalmente para la creación de los Pokémon base con sus atributos y comportamientos necesarios. Las clases que se crearán serán las siguientes:

Clase Pokemon

La clase Pokemon representa a un Pokémon con atributos que describen sus características principales.

Atributos:

nombre: Nombre del Pokémon.

tipo: Tipo del Pokémon, representado como un enum Tipo que puede ser NORMAL, FUEGO o AGUA.

vida, ataque, defensa, ataqueEspecial, defensaEspecial: Estadísticas de batalla del Pokémon, que incluyen puntos de vida, ataques físico y especial, y defensas física y especial.

Constructor (Pokemon(String nombre, Tipo tipo)):

- Inicializa un Pokémon con un nombre y un tipo específico.
- Llama al método establecerAtributos para configurar las estadísticas del Pokémon según su tipo.

Métodos establecerAtributos:

Utiliza un switch para asignar valores específicos a las estadísticas dependiendo del tipo de Pokémon.

Métodos getters (getNombre(), getTipo(), etc.):

Permiten acceder a los atributos privados del Pokémon desde fuera de la clase.

Método toString():

Sobrescribe el método toString() para proporcionar una representación en cadena del Pokémon, mostrando su nombre, tipo y estadísticas de batalla.

Clase NodoPokemon

La clase NodoPokemon representa un nodo que contiene un Pokémon y se utiliza en estructuras de datos enlazadas, como una pila en este caso.

Atributos:

elemento: El Pokémon almacenado en este nodo.

siguiente: Referencia al siguiente nodo en la estructura de datos enlazada.

Constructor por defecto (NodoPokemon()):

Inicializa un nodo sin ningún Pokémon asociado y establece siguiente como null.

Métodos getters y setters (getElemento(), setElemento(), getSiguiente(), setSiguiente()):

Permiten acceder y modificar los atributos privados del nodo desde fuera de la clase.

Método toString():

Sobrescribe el método toString() para proporcionar una representación en cadena del Pokémon almacenado en el nodo. Este método delega la representación en cadena al método toString() de la clase Pokemon.

Clase PilaPokemon

La clase PilaPokemon implementa una pila de Pokémon utilizando nodos de tipo NodoPokemon.

Atributos:

cima: Referencia al nodo superior (último nodo apilado) de la pila.

Método esVacia():

Verifica si la pila está vacía comprobando si cima es null.

Método apilar(Pokemon.Tipo tipo, String nombre):

- Crea un nuevo nodo NodoPokemon con un Pokémon recién creado usando el tipo y nombre proporcionados.
- Si la pila está vacía, establece el nuevo nodo como la cima.
- Si la pila no está vacía, coloca el nuevo nodo encima del nodo actual (cima).

Método desapilar():

- Elimina y devuelve el Pokémon en la cima de la pila.
- Actualiza cima para que apunte al siguiente nodo en la pila.

Método toString():

Proporcionar una representación en cadena de todos los Pokémon apilados en la pila.

Propuesta de manejo para los jugadores

Clases Principales

Clase Aprendiz:

Propósito: Representa a un personaje que participa en las batallas.

Atributos: Contiene atributos como nombre, puntos de vida (HP), ataque (ATK), defensa (DEF), velocidad (SPD), una lista de habilidades y un inventario de ítems.

Métodos: Incluye métodos para realizar acciones como atacar, defender, usar habilidades, usar ítems y cambiar de Aprendiz.

Clase Habilidad:

Propósito: Representa una habilidad especial que un Aprendiz puede usar en batalla.

Atributos: Incluye nombre, descripción, efecto y costo de la habilidad.

Métodos: Métodos para aplicar la habilidad en el contexto de la batalla.

Clase Item:

Propósito: Representa un ítem que puede ser usado en la batalla, como pociones o mejoras de estadísticas.

Atributos: Contiene nombre, tipo y efecto del ítem.

Métodos: Métodos para usar el ítem y aplicar su efecto.

Clase Batalla:

Propósito: Maneja la lógica de la batalla, incluyendo la determinación del orden de turnos y la ejecución de acciones.

Atributos: Contiene las listas de Aprendices de ambos equipos y el orden de turnos.

Métodos: Incluye métodos para iniciar la batalla, determinar el orden de turnos, ejecutar acciones y finalizar la batalla.

Clase Equipo:

Propósito: Representa un equipo de Aprendices.

Atributos: Contiene una lista de Aprendices.

Métodos: Métodos para gestionar el equipo, como agregar o remover Aprendices y obtener el estado del equipo.

Clase Juego:

Propósito: Controla el flujo general del juego, incluyendo la inicialización de los equipos y el inicio de las batallas.

Atributos: Contiene los equipos participantes.

Métodos: Incluye métodos para iniciar el juego y gestionar el flujo de las batallas.

Funcionamiento

Iniciar el Juego:

La clase Juego inicializa los equipos y sus Aprendices.

Se crean instancias de Aprendiz, Habilidad e Item y se asignan a los Aprendices correspondientes.

El juego llama al método de iniciar batalla en la clase Batalla.

Iniciar la Batalla:

La clase Batalla recibe las listas de Aprendices de ambos equipos.

Determina el orden de turnos basado en la velocidad (SPD) de cada Aprendiz. En caso de empate, se decide aleatoriamente.

Ejecución de Turnos:

La clase Batalla itera sobre la lista de Aprendices en el orden determinado.

Cada Aprendiz ejecuta su acción (atacar, defender, usar habilidad, usar ítem, cambiar de Aprendiz) durante su turno.

Las acciones se ejecutan y se actualizan los estados de los Aprendices (por ejemplo, reducir HP del objetivo si es un ataque).

Finalización de la Batalla:

La clase Batalla verifica si uno de los equipos ha sido derrotado (todos los Aprendices con HP = 0).

Si la batalla ha terminado, se realizan las acciones finales como asignar experiencia (EXP) a los Aprendices supervivientes y actualizar sus niveles si corresponde.

Progreso del Aprendiz:

Los Aprendices ganan experiencia (EXP) al participar en la batalla.

Al acumular suficiente EXP, un Aprendiz sube de nivel, incrementando sus atributos básicos y aprendiendo nuevas habilidades.

Estrategia del Jugador:

Los jugadores deben formar equipos con Aprendices que tengan habilidades complementarias.

Durante la batalla, deben adaptarse a las situaciones cambiantes y gestionar cuidadosamente sus recursos (ítems y habilidades especiales).

Interacción entre Clases

La clase Juego coordina la inicialización de los equipos y el inicio de la batalla.

La clase Batalla gestiona la lógica de la batalla, incluyendo el orden de turnos y la ejecución de acciones.

Las clases Aprendiz, Habilidad e Item encapsulan la lógica específica de cada entidad y se utilizan dentro de la clase Batalla para ejecutar las acciones.

La clase Equipo organiza y maneja los Aprendices de cada equipo, proporcionando información a la clase Batalla.

Creación de y uso de repositorio en Github

- https://github.com/Knowleach314/Estructura-de-Datos-.git